



Ahsanullah University of Science and Technology

Department of Electrical and Electronic Engineering

EEE 3218: Digital Signal Processing Lab

OPEN-ENDED LABORATORY REPORT

Fall 2025

Recreation of a Phase-Sensitive Dynamic Filter (PSDF)

Reproduced paper

“Phase-Sensitive Dynamic Filters: A Conceptual Model for Energy-Preserving, Instantaneously Tunable, Symbol-Level Signal Processing Using Complex IIR Filters” S. C. Chen et al., IEEE Access, vol. 14, 2026

Group Members

Name	Student ID
Karib Muhammad Shayek	20230105164
Tasin Hasan Bani	20230105166
Fazlay Elahi	20220105234
Jannatul Ferdous Ovy	20230105177

1. Abstract

This report is a reproduction and critical review of a recent IEEE Access paper on Phase-Sensitive Dynamic Filters (PSDFs). A PSDF is a time-varying, complex-coefficient IIR filter that is switched in step with the symbol clock, so that frequency- or phase-modulated signals can be filtered without throwing away the filter's stored energy. We wrote one compact Python script that rebuilds the paper's main theory and time-domain figures from the equations: the low-pass, bandpass, high-pass and bandstop responses (paper Figs. 3 and 10), the QPSK phase-nullification demo (Fig. 5), the continuous charging and decaying of the dynamic bandpass and notch filters (Figs. 8 and 12), and the BFSK and QPSK comparisons against switched or fixed filters (Figs. 13, 14, 16 and 17). Every signal in the script is noise-free and analytic, so whatever we see in the plots comes only from the filter recursion itself.

The results support the paper's main claim. If only the *phase* of the pole is changed and its magnitude is left alone, a single-pole complex IIR filter can follow a new carrier frequency or symbol phase and still keep its accumulated state. In numbers:

- At BFSK symbol boundaries the dynamic bandpass envelope changed by only 0.12% on average (0.52% at worst). A switched filter that restarts every symbol lost about 99.3% of its envelope.
- After 70 μs of QPSK the dynamic filter reached an envelope of 0.9988, which equals the analytic value $1 - e^{-t/\tau}$. The fixed filter without phase correction stayed near 0.14, with a coefficient of variation of about 0.41.
- The dynamic notch filters suppressed the tracked signal by about 19.5 dB for BFSK (after 10 symbols) and about 57.9 dB for QPSK (after 28 symbols). The switched/fixed notch filters gave almost no suppression.
- Setting the pole magnitude directly from the time constant, $|b| = e^{-1/(\tau f_s)}$, reproduces the paper's 31.2 μs and 10.4 μs exactly. It also shows that the paper's bandwidths (10 kHz and 30 kHz) are edge-to-edge values, about twice $1/(2\pi\tau)$.

The report ends with a discussion of what the model and our reproduction leave out: noise, spectra, the AM-interferer test and the full-duplex study are not part of this version of the script.

2. Introduction

Wireless systems today use a lot of frequency-hopped and phase-modulated waveforms, and these are hard to handle with ordinary fixed RF filters. If the filter is wide enough to cover every frequency or phase the signal uses, it also lets in extra out-of-band noise. If it is narrow, it needs more time to settle than one symbol lasts, and the symbol transitions get distorted. The classical relation that ties these two together is $\Delta f\tau = 1/(2\pi)$: a narrower bandwidth (higher Q) always means a longer time constant.

The paper proposes an idealized way around this. A Phase-Sensitive Dynamic Filter is a single-pole complex IIR filter whose coefficients are changed together with the symbol clock. The filter's internal state, its "charge", is never cleared when the coefficients change, and only the *phase* of the coefficient moves, not its magnitude. So the filter keeps a long time constant (a narrow noise bandwidth) and still tracks frequency and phase changes that are faster than that time constant would normally allow. The catch is that the receiver must know the transmitted symbol sequence in advance.

For this project we built the model in Python around one general routine: a time-varying complex single-pole filter in which the feed-forward weights, feedback coefficient, output phase restoration and state-reset period are all given as per-sample arrays. Bandpass, notch, switched and dynamic filters are all produced by the same routine, just by changing those arrays. This keeps the link between the paper's equations (4)–(29) and

the plotted waveforms easy to follow, and lets us check the claims of frequency tunability, phase tunability and notch duality in a single implementation.

3. Reproduced Segment

3.1 Source Paper Overview

Paper: S. C. Chen, K. D. Bhakta, I. P. Roberts and Y. E. Wang, “Phase-Sensitive Dynamic Filters: A Conceptual Model for Energy-Preserving, Instantaneously Tunable, Symbol-Level Signal Processing Using Complex IIR Filters,” *IEEE Access*, vol. 14, pp. 61414–61429, 2026, DOI: 10.1109/ACCESS.2026.3685577.

Problem addressed: a conventional tunable RF bandpass filter cannot move its passband faster than about its own settling time, and that settling time is inversely proportional to bandwidth. Because of this, narrowband, real-time, symbol-level filtering of frequency- or phase-hopped signals is difficult without either distorting the signal or admitting extra noise.

Method: a real single-pole low-pass IIR prototype is turned into a complex bandpass filter by multiplying its feedback coefficient by a unit-magnitude complex exponential at the wanted centre frequency. Since only the phase of the coefficient changes, the pole keeps the same distance from the origin and the time constant does not change when the filter is retuned. A similar feed-forward step removes the input phase and puts it back later, which handles phase-shift-keyed signals. Applying the same shift to a high-pass prototype gives a dynamic notch (bandstop) filter. The paper checks the model with BFSK and QPSK simulations, an amplitude-modulated interferer test and a full-duplex self-interference case study.

3.2 Scope of This Reproduction

The script regenerates the figures listed in Table 1. Wherever the paper gives its own parameters (sampling rates, carrier frequencies, symbol periods, time constants) we used them.

Table 1. Mapping between the paper’s figures and the figures in this report.

Paper Fig.	What it shows	Figure in this report
3	Real low-pass, real bandpass and shifted complex bandpass responses	Fig. 1
10	Real high-pass, real bandstop and shifted complex bandstop (notch) responses	Fig. 2
5	QPSK phase nullification and correction (waveform and constellation)	Figs. 3 and 4
8	Continuous charging of the dynamic bandpass filter (BFSK, QPSK)	Fig. 5
12	Continuous decaying of the dynamic notch filter (BFSK, QPSK)	Fig. 6
13	BFSK: switched vs. dynamic bandpass, time domain	Fig. 7
14	BFSK: switched vs. dynamic notch, time domain	Fig. 8
16	QPSK: fixed vs. dynamic bandpass, time domain	Fig. 9
17	QPSK: fixed vs. dynamic notch, time domain	Fig. 10

Not covered: the noisy-signal spectra (paper Figs. 15 and 18), the in-band AM-interferer test (Fig. 19), the full-duplex co-site leakage simulation (Fig. 21) and the mismatch-sensitivity curve (Fig. 22). The script has no random noise and no FFT-based spectra, so these are listed as limitations in Section 5.

3.3 Theoretical Model Reproduced

The starting point is the classical real single-pole IIR low-pass filter (paper Eqs. 4–6):

$$y[n] = ax[n] + by[n-1], \quad b = e^{-2\pi f_{\text{cutoff}}}, \quad a = 1 - b$$

Replacing the delay operator in $H(z) = a/(1 - bz^{-1})$ with a frequency-shifted one swaps the real feedback coefficient b for $b' = be^{j2\pi f_c/f_s}$. This moves the whole low-pass response to the centre frequency f_c , while $|b'|$ (and so the settling behaviour) stays the same (Eqs. 9–10). Doing the same to the two-tap high-pass prototype ($a_0 = (1+d)/2$, $a_1 = -(1+d)/2$, $b_1 = d$) gives a complex bandstop (notch) filter (Eqs. 21–26).

For dynamic frequency tunability, b' is switched between complex values at the symbol boundaries while the recursive state is carried forward. For dynamic phase tunability, the feed-forward coefficient is changed to $a' = ae^{-j\varphi}$, which cancels the instantaneous symbol phase φ before the signal enters the recursion. After filtering, the phase is put back by multiplying with $e^{j\varphi}$ and the real part is taken as the output (Eqs. 12–16). The time constant follows from

$$|b| = e^{-1/d}, \quad d = \tau f_s \text{ (time constant in samples)} \quad (\text{Eq. 17})$$

Implementation idea. All filter variants are made by one routine, `time_varying_filter(INPUT, CURRENT_WEIGHT, PREVIOUS_WEIGHT, FEEDBACK, RESTORE, RESET_PERIOD)`, which computes

$$\begin{aligned} \text{state}[n] &= \text{CURRENT_WEIGHT}[n]x[n] + \text{PREVIOUS_WEIGHT}[n]x[n-1] + \text{FEEDBACK}[n]\text{state}[n-1] \\ y[n] &= \text{Re}\{ \text{RESTORE}[n]\text{state}[n] \} \end{aligned}$$

and zeros the state and the previous input every `RESET_PERIOD` samples. With `PREVIOUS_WEIGHT` = 0 it is a bandpass filter. With `CURRENT_WEIGHT` = $(1+b)/2$ and `PREVIOUS_WEIGHT` = $-(1+b)/2e^{j\omega}$ it is a notch. Setting `RESET_PERIOD` equal to the symbol length gives a switched filter that restarts every symbol, and setting it equal to the signal length gives a dynamic filter that is never reset.

3.4 Simulation Methodology

The script runs these blocks in order:

- **Frequency responses (Figs. 1, 2).** The impulse responses of the real low-pass, the shifted complex bandpass ($be^{j\omega}$), its mirror ($be^{-j\omega}$) and their sum (the traditional real bandpass) are computed with `scipy.signal.filter` and a 2048-point FFT, using $f_s = 1$ MHz, normalized cutoff 0.005 (5 kHz) and centre frequency 100 kHz. The high-pass, the positive and negative notches and their product (the real bandstop) are obtained the same way.
- **QPSK phase nullification (Figs. 3, 4).** A 28-symbol QPSK signal ($f_c = 1$ MHz, phases $45^\circ, 135^\circ, 225^\circ, 315^\circ$) is multiplied by $e^{-j\varphi}$ and then by $e^{+j\varphi}$, and the constellation is plotted before, after nullification and after restoration.
- **BFSK signal (Figs. 5–8).** 10 symbols (sequence 0100110101), $f = 1$ MHz / 0.5 MHz, $T_s = 10$ μ s, $f_s = 16$ Msps, $\tau = 31.2$ μ s (about 3.1 symbol periods). Four filters are run: dynamic bandpass, switched bandpass (state reset every symbol), dynamic notch and switched notch.
- **QPSK signal (Figs. 5, 6, 9, 10).** 28 symbols, $f_c = 1$ MHz, $T_s = 2.5$ μ s, $f_s = 32$ Msps, $\tau = 10.4$ μ s (about 4.2 symbol periods). The dynamic filters use phase removal and restoration. The comparison filters have no phase correction and no reset, so they act as a fixed narrowband filter.

Since the input signals are ideal and noise-free, each filter output depends only on its own recursion, and the charging and decaying envelopes can be predicted exactly from the time constant. We use this for validation in Section 4.

3.5 Code for Simulation

The complete script is given below. It is also supplied as a separate file (DSP_OEL_CODE.py).

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

def time_varying_filter(INPUT, CURRENT_WEIGHT, PREVIOUS_WEIGHT, FEEDBACK, RESTORE, RESET_PERIOD):
    OUTPUT = np.zeros(len(INPUT))
    STATE = 0
    PREVIOUS_INPUT = 0
    for INDEX in range(len(INPUT)):
        if INDEX % RESET_PERIOD == 0:
            STATE = 0
            PREVIOUS_INPUT = 0
        STATE = (CURRENT_WEIGHT[INDEX]*INPUT[INDEX]
                + PREVIOUS_WEIGHT[INDEX]*PREVIOUS_INPUT
                + FEEDBACK[INDEX]*STATE)
        PREVIOUS_INPUT = INPUT[INDEX]
        OUTPUT[INDEX] = np.real(RESTORE[INDEX]*STATE)
    return OUTPUT

def plot_signal(ROWS, POSITION, TIME, SIGNAL, COLOR, TITLE):
    plt.subplot(ROWS, 1, POSITION)
    plt.plot(TIME, SIGNAL, COLOR)
    plt.title(TITLE)
    plt.xlabel('Time (s)')
    plt.grid(True)

# FIGURES 3 AND 10 : FREQUENCY RESPONSES
SAMPLING_FREQUENCY = 1e6
NORMALIZED_CUTOFF = 0.005
CENTER_FREQUENCY = 100e3
POINTS = 2048
FEEDBACK = np.exp(-2*np.pi*NORMALIZED_CUTOFF)
FEEDFORWARD = 1 - FEEDBACK
ROTATION = np.exp(1j*2*np.pi*CENTER_FREQUENCY/SAMPLING_FREQUENCY)
IMPULSE = np.zeros(POINTS)
IMPULSE[0] = 1
FREQUENCY = np.fft.fftshift(np.fft.fftfreq(POINTS, d=1/SAMPLING_FREQUENCY))/1e3
LOWPASS = np.fft.fft(signal.lfilter([FEEDFORWARD], [1, -FEEDBACK], IMPULSE))
POSITIVE_BANDPASS = np.fft.fft(signal.lfilter([FEEDFORWARD], [1, -FEEDBACK*ROTATION], IMPULSE))
NEGATIVE_BANDPASS = np.fft.fft(signal.lfilter([FEEDFORWARD], [1, -FEEDBACK/ROTATION], IMPULSE))

plt.figure(figsize=(8, 8))
plt.subplot(3, 1, 1)
plt.plot(FREQUENCY, np.fft.fftshift(np.abs(LOWPASS)))
plt.title('IIR Lowpass Filter')
plt.subplot(3, 1, 2)
plt.plot(FREQUENCY, np.fft.fftshift(np.abs(POSITIVE_BANDPASS + NEGATIVE_BANDPASS)))
plt.title('Traditional Real IIR Bandpass Filter')
plt.subplot(3, 1, 3)
plt.plot(FREQUENCY, np.fft.fftshift(np.abs(POSITIVE_BANDPASS)))
plt.title('Shifted Complex IIR Bandpass Filter')
for POSITION in range(1, 4):
    plt.subplot(3, 1, POSITION)
    plt.xlabel('Frequency (kHz)')
    plt.grid(True)
plt.tight_layout()
```

```

NUMERATOR = (1 + FEEDBACK)/2*np.array([1, -1])
HIGHPASS = np.fft.fft(signal.lfilter(NUMERATOR, [1, -FEEDBACK], IMPULSE))
POSITIVE_NOTCH = np.fft.fft(signal.lfilter(NUMERATOR*[1, ROTATION], [1, -FEEDBACK*ROTATION],
IMPULSE))
NEGATIVE_NOTCH = np.fft.fft(signal.lfilter(NUMERATOR/[1, ROTATION], [1, -FEEDBACK/ROTATION],
IMPULSE))

plt.figure(figsize=(8, 8))
plt.subplot(3, 1, 1)
plt.plot(FREQUENCY, np.fft.fftshift(np.abs(HIGHPASS)))
plt.title('IIR Highpass Filter')
plt.subplot(3, 1, 2)
plt.plot(FREQUENCY, np.fft.fftshift(np.abs(POSITIVE_NOTCH*NEGATIVE_NOTCH)))
plt.title('Traditional Real IIR Bandstop Filter')
plt.subplot(3, 1, 3)
plt.plot(FREQUENCY, np.fft.fftshift(np.abs(POSITIVE_NOTCH)))
plt.title('Shifted Complex IIR Bandstop Filter')
for POSITION in range(1, 4):
    plt.subplot(3, 1, POSITION)
    plt.xlabel('Frequency (kHz)')
    plt.grid(True)
plt.tight_layout()

# BFSK AND QPSK SIGNALS AND FILTERS
BFSK_SAMPLING_FREQUENCY = 16e6
BFSK_SYMBOL_PERIOD = 10e-6
BFSK_TIME_CONSTANT = 31.2e-6
BFSK_FREQUENCIES = np.array([1e6, 0.5e6])
BFSK_SEQUENCE = np.array([0, 1, 0, 0, 1, 1, 0, 1, 0, 1])
BFSK_SAMPLES_PER_SYMBOL = int(np.round(BFSK_SAMPLING_FREQUENCY*BFSK_SYMBOL_PERIOD))
BFSK_TIME = np.arange(len(BFSK_SEQUENCE)*BFSK_SAMPLES_PER_SYMBOL)/BFSK_SAMPLING_FREQUENCY
BFSK_FREQUENCY_PER_SAMPLE =
BFSK_FREQUENCIES[BFSK_SEQUENCE[np.arange(len(BFSK_TIME))/BFSK_SAMPLES_PER_SYMBOL]]
BFSK_ANALYTIC = np.exp(1j*2*np.pi*BFSK_FREQUENCY_PER_SAMPLE*BFSK_TIME)
BFSK_ROTATION = np.exp(1j*2*np.pi*BFSK_FREQUENCY_PER_SAMPLE/BFSK_SAMPLING_FREQUENCY)
BFSK_FEEDBACK = np.exp(-1/(BFSK_TIME_CONSTANT*BFSK_SAMPLING_FREQUENCY))
BFSK_ONES = np.ones(len(BFSK_TIME))
BFSK_ZEROS = np.zeros(len(BFSK_TIME))
BFSK_LENGTH = len(BFSK_TIME)
BFSK_PASS_WEIGHT = (1 - BFSK_FEEDBACK)*BFSK_ONES
BFSK_NOTCH_WEIGHT = (1 + BFSK_FEEDBACK)/2*BFSK_ONES
BFSK_ORIGINAL = np.real(BFSK_ANALYTIC)

BFSK_DYNAMIC_PASS = time_varying_filter(
    BFSK_ANALYTIC, BFSK_PASS_WEIGHT, BFSK_ZEROS,
    BFSK_FEEDBACK*BFSK_ROTATION, BFSK_ONES, BFSK_LENGTH)
BFSK_SWITCHED_PASS = time_varying_filter(
    BFSK_ANALYTIC, BFSK_PASS_WEIGHT, BFSK_ZEROS,
    BFSK_FEEDBACK*BFSK_ROTATION, BFSK_ONES, BFSK_SAMPLES_PER_SYMBOL)
BFSK_DYNAMIC_NOTCH = time_varying_filter(
    BFSK_ANALYTIC, BFSK_NOTCH_WEIGHT, -BFSK_NOTCH_WEIGHT*BFSK_ROTATION,
    BFSK_FEEDBACK*BFSK_ROTATION, BFSK_ONES, BFSK_LENGTH)
BFSK_SWITCHED_NOTCH = time_varying_filter(
    BFSK_ANALYTIC, BFSK_NOTCH_WEIGHT, -BFSK_NOTCH_WEIGHT*BFSK_ROTATION,
    BFSK_FEEDBACK*BFSK_ROTATION, BFSK_ONES, BFSK_SAMPLES_PER_SYMBOL)

QPSK_SAMPLING_FREQUENCY = 32e6
QPSK_SYMBOL_PERIOD = 2.5e-6
QPSK_TIME_CONSTANT = 10.4e-6
QPSK_CARRIER_FREQUENCY = 1e6
QPSK_PHASES = np.array([45, 135, 225, 315])*np.pi/180
QPSK_SEQUENCE = np.array([0, 2, 1, 3, 0, 1, 2, 3, 1, 0, 3, 2, 0, 1,

```

```

        3, 2, 1, 0, 2, 3, 2, 0, 3, 1, 0, 3, 1, 2])
QPSK_SAMPLES_PER_SYMBOL = int(np.round(QPSK_SAMPLING_FREQUENCY*QPSK_SYMBOL_PERIOD))
QPSK_TIME = np.arange(len(QPSK_SEQUENCE)*QPSK_SAMPLES_PER_SYMBOL)/QPSK_SAMPLING_FREQUENCY
QPSK_PHASE_PER_SAMPLE =
QPSK_PHASES[QPSK_SEQUENCE[np.arange(len(QPSK_TIME))/QPSK_SAMPLES_PER_SYMBOL]]
QPSK_ANALYTIC = np.exp(1j*(2*np.pi*QPSK_CARRIER_FREQUENCY*QPSK_TIME + QPSK_PHASE_PER_SAMPLE))
QPSK_ROTATION = np.exp(1j*2*np.pi*QPSK_CARRIER_FREQUENCY/QPSK_SAMPLING_FREQUENCY)
QPSK_FEEDBACK = np.exp(-1/(QPSK_TIME_CONSTANT*QPSK_SAMPLING_FREQUENCY))
QPSK_ONES = np.ones(len(QPSK_TIME))
QPSK_ZEROS = np.zeros(len(QPSK_TIME))
QPSK_LENGTH = len(QPSK_TIME)
QPSK_PHASE_REMOVAL = np.exp(-1j*QPSK_PHASE_PER_SAMPLE)
QPSK_PHASE_RESTORE = np.exp(1j*QPSK_PHASE_PER_SAMPLE)
QPSK_PASS_WEIGHT = (1 - QPSK_FEEDBACK)*QPSK_ONES
QPSK_NOTCH_WEIGHT = (1 + QPSK_FEEDBACK)/2*QPSK_ONES
QPSK_ORIGINAL = np.real(QPSK_ANALYTIC)

QPSK_DYNAMIC_PASS = time_varying_filter(
    QPSK_ANALYTIC*QPSK_PHASE_REMOVAL, QPSK_PASS_WEIGHT, QPSK_ZEROS,
    QPSK_FEEDBACK*QPSK_ROTATION*QPSK_ONES, QPSK_PHASE_RESTORE, QPSK_LENGTH)
QPSK_SWITCHED_PASS = time_varying_filter(
    QPSK_ANALYTIC, QPSK_PASS_WEIGHT, QPSK_ZEROS,
    QPSK_FEEDBACK*QPSK_ROTATION*QPSK_ONES, QPSK_ONES, QPSK_LENGTH)
QPSK_DYNAMIC_NOTCH = time_varying_filter(
    QPSK_ANALYTIC*QPSK_PHASE_REMOVAL, QPSK_NOTCH_WEIGHT, -QPSK_NOTCH_WEIGHT*QPSK_ROTATION,
    QPSK_FEEDBACK*QPSK_ROTATION*QPSK_ONES, QPSK_PHASE_RESTORE, QPSK_LENGTH)
QPSK_SWITCHED_NOTCH = time_varying_filter(
    QPSK_ANALYTIC, QPSK_NOTCH_WEIGHT, -QPSK_NOTCH_WEIGHT*QPSK_ROTATION,
    QPSK_FEEDBACK*QPSK_ROTATION*QPSK_ONES, QPSK_ONES, QPSK_LENGTH)

# FIGURE 5 : PHASE NULLIFICATION OF QPSK
PHASE_FREE = np.real(QPSK_ANALYTIC*QPSK_PHASE_REMOVAL)
RESTORED = np.real(QPSK_ANALYTIC*QPSK_PHASE_REMOVAL*QPSK_PHASE_RESTORE)
ANGLE = np.arange(0, 2*np.pi, 0.01)

plt.figure(figsize=(10, 8))
plot_signal(3, 1, QPSK_TIME[1600:2240], QPSK_ORIGINAL[1600:2240], 'r', 'Original QPSK Signal')
plot_signal(3, 2, QPSK_TIME[1600:2240], PHASE_FREE[1600:2240], 'b', 'Phase-Nullified QPSK
Signal')
plot_signal(3, 3, QPSK_TIME[1600:2240], RESTORED[1600:2240], 'm', 'Phase-Corrected QPSK Signal')
plt.tight_layout()

plt.figure(figsize=(6, 10))
plt.subplot(3, 1, 1)
plt.plot(np.cos(ANGLE), np.sin(ANGLE), 'k:')
plt.plot(np.cos(QPSK_PHASES), np.sin(QPSK_PHASES), 'bo')
plt.subplot(3, 1, 2)
plt.plot(np.cos(ANGLE), np.sin(ANGLE), 'k:')
plt.plot(1, 0, 'bo')
plt.subplot(3, 1, 3)
plt.plot(np.cos(ANGLE), np.sin(ANGLE), 'k:')
plt.plot(np.cos(QPSK_PHASES), np.sin(QPSK_PHASES), 'bo')
for POSITION in range(1, 4):
    plt.subplot(3, 1, POSITION)
    plt.axhline(0, color='k')
    plt.axvline(0, color='k')
    plt.xlabel('Re')
    plt.ylabel('Im')
plt.tight_layout()

# FIGURE 8 : CONTINUOUS CHARGING
plt.figure(figsize=(8, 10))
plot_signal(4, 1, BFSK_TIME, BFSK_ORIGINAL, 'b', 'Original Signal - BFSK')

```

```

plot_signal(4, 2, BFSK_TIME, BFSK_DYNAMIC_PASS, 'b', 'Dynamically Filtered Signal - BFSK')
plot_signal(4, 3, QPSK_TIME[:640], QPSK_ORIGINAL[:640], 'b', 'Original Signal - QPSK')
plot_signal(4, 4, QPSK_TIME[:640], QPSK_DYNAMIC_PASS[:640], 'b', 'Dynamically Filtered Signal - QPSK')
plt.tight_layout()

# FIGURE 12 : CONTINUOUS DECAY
plt.figure(figsize=(8, 10))
plot_signal(4, 1, BFSK_TIME, BFSK_ORIGINAL, 'b', 'Original Signal - BFSK')
plot_signal(4, 2, BFSK_TIME, BFSK_DYNAMIC_NOTCH, 'b', 'Dynamically Notch Filtered Signal - BFSK')
plot_signal(4, 3, QPSK_TIME[:640], QPSK_ORIGINAL[:640], 'b', 'Original Signal - QPSK')
plot_signal(4, 4, QPSK_TIME[:640], QPSK_DYNAMIC_NOTCH[:640], 'b', 'Dynamically Notch Filtered Signal - QPSK')
plt.tight_layout()

# FIGURES 13, 14, 16, 17 : SWITCHED VERSUS DYNAMIC
plt.figure(figsize=(8, 8))
plot_signal(3, 1, BFSK_TIME, BFSK_ORIGINAL, 'b', 'Original Signal - BFSK')
plot_signal(3, 2, BFSK_TIME, BFSK_SWITCHED_PASS, 'k', 'Switched Passband Signal - BFSK')
plot_signal(3, 3, BFSK_TIME, BFSK_DYNAMIC_PASS, 'r', 'Dynamically Filtered Signal - BFSK')
plt.tight_layout()

plt.figure(figsize=(8, 8))
plot_signal(3, 1, BFSK_TIME, BFSK_ORIGINAL, 'b', 'Original Signal - BFSK')
plot_signal(3, 2, BFSK_TIME, BFSK_SWITCHED_NOTCH, 'k', 'Switched Stopband Signal - BFSK')
plot_signal(3, 3, BFSK_TIME, BFSK_DYNAMIC_NOTCH, 'r', 'Dynamically Notch Filtered Signal - BFSK')
plt.tight_layout()

plt.figure(figsize=(8, 8))
plot_signal(3, 1, QPSK_TIME[:1600], QPSK_ORIGINAL[:1600], 'b', 'Original Signal - QPSK')
plot_signal(3, 2, QPSK_TIME[:1600], QPSK_SWITCHED_PASS[:1600], 'k', 'Switched Passband Signal - QPSK')
plot_signal(3, 3, QPSK_TIME[:1600], QPSK_DYNAMIC_PASS[:1600], 'r', 'Dynamically Filtered Signal - QPSK')
plt.tight_layout()

plt.figure(figsize=(8, 8))
plot_signal(3, 1, QPSK_TIME[:1600], QPSK_ORIGINAL[:1600], 'b', 'Original Signal - QPSK')
plot_signal(3, 2, QPSK_TIME[:1600], QPSK_SWITCHED_NOTCH[:1600], 'k', 'Switched Stopband Signal - QPSK')
plot_signal(3, 3, QPSK_TIME[:1600], QPSK_DYNAMIC_NOTCH[:1600], 'r', 'Dynamically Notch Filtered Signal - QPSK')
plt.tight_layout()

plt.show()

```

4. Results and Analysis

4.1 Complex Filter Frequency Responses

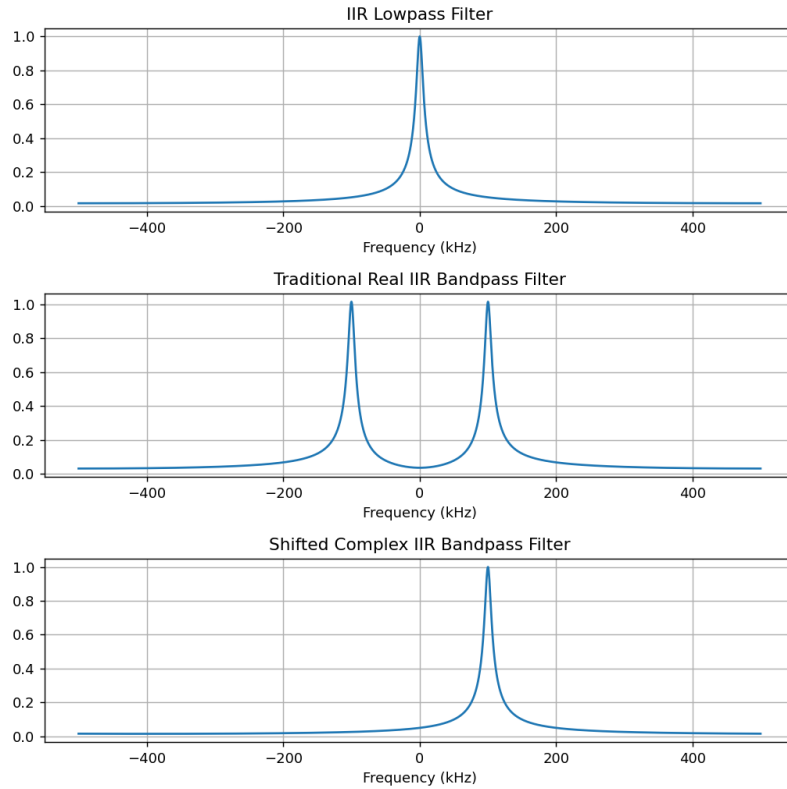


Figure 1. Reproduced paper Fig. 3: IIR low-pass, traditional real bandpass and shifted complex bandpass responses (normalized cutoff $0.005 = 5$ kHz, shifted to 100 kHz, $f_s = 1$ MHz).

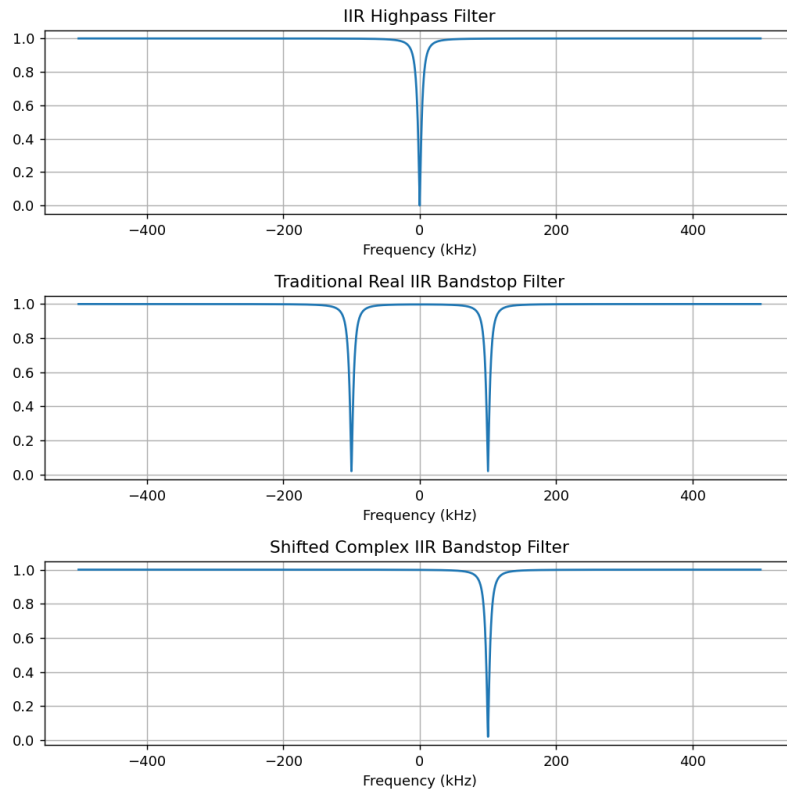


Figure 2. Reproduced paper Fig. 10: IIR high-pass, traditional real bandstop and shifted complex bandstop (notch) responses for the same parameters.

Both figures show the paper’s central geometric argument. The traditional real bandpass/stopband filter has mirror-image responses at $+100$ kHz and -100 kHz, while the complex (shifted) version keeps only one passband or stopband, at $+100$ kHz. Numerically, the shifted complex bandpass peaks at 100.1 kHz (the FFT bin spacing is 0.49 kHz) with unity gain and has a -3 dB width of 9.8 kHz. This is twice the 4.9 kHz half-width of the low-pass prototype, which agrees with the paper’s statement that the prototype cutoff is half the edge-to-edge bandwidth. Its gain at -100 kHz is only 0.027 (-31 dB). The complex notch reaches a minimum of 0.020 at 100.1 kHz and returns to 1.00 at the band edges, while the real bandstop has nulls at both ± 100 kHz. The asymmetry of the complex filter about 0 Hz is exactly what we expect from a coefficient that is not conjugate-symmetric.

4.2 QPSK Phase-Nullification Demonstration

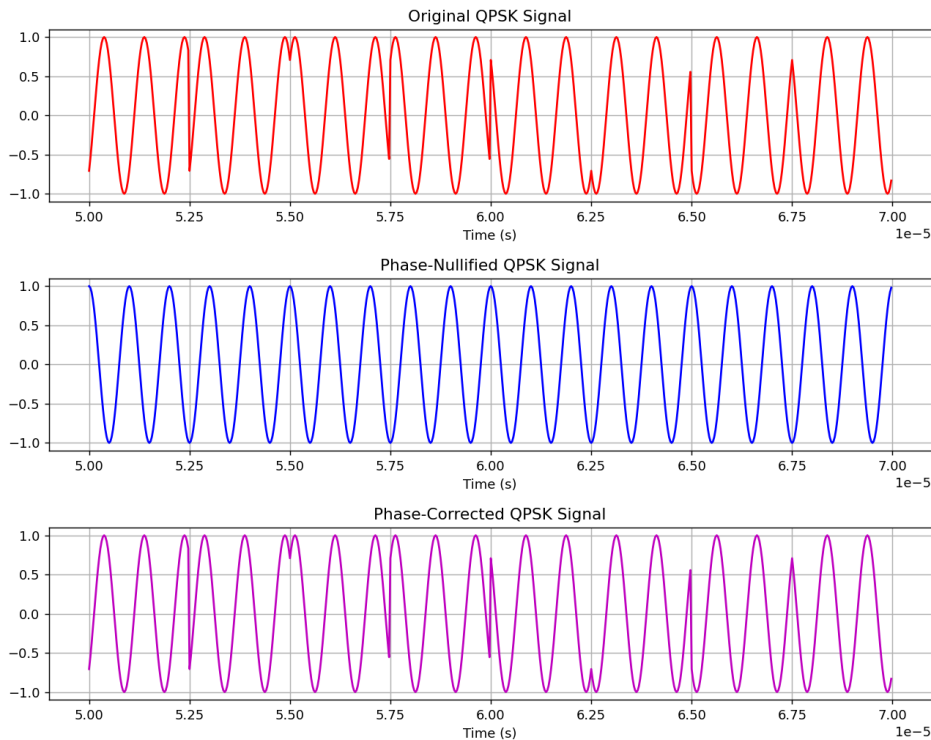


Figure 3. Reproduced paper Fig. 5 (waveforms): original QPSK signal (top), phase-nullified signal (middle) and phase-corrected signal (bottom), $50\text{--}70\text{ }\mu\text{s}$ window.

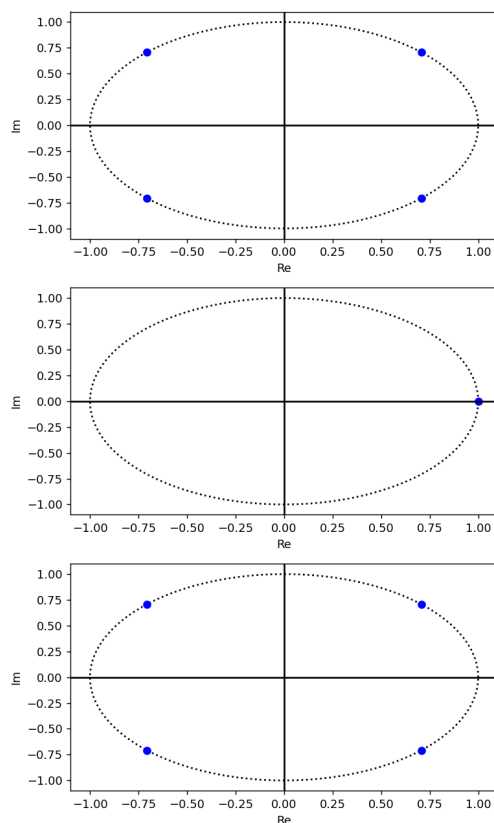


Figure 4. Reproduced paper Fig. 5 (constellations): the four QPSK points (top) collapse to one point at $(1, 0)$ after nullification (middle) and are restored (bottom).

Multiplying the analytic signal by $e^{-j\varphi}$ before filtering moves all four QPSK phases onto the positive real axis. After the phase is removed, the signal equals exactly $1 + 0j$ at every sample, so the filter sees an unmodulated sinusoid. This is the condition under which a narrowband filter no longer distorts the phase transitions. Applying $e^{+j\varphi}$ again restores the original four-point constellation with a maximum reconstruction error of 2.2×10^{-16} , which is just floating-point round-off. The nullify/restore pair is therefore lossless by construction.

4.3 Continuous Charging vs. Continuous Decaying

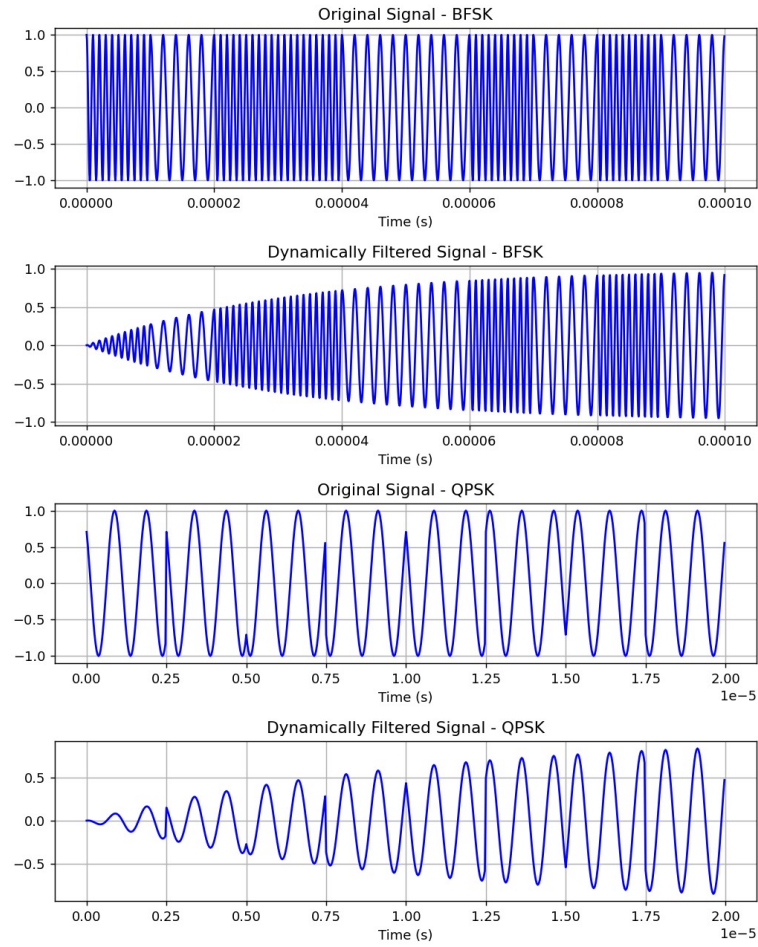


Figure 5. Reproduced paper Fig. 8: the dynamic bandpass filter charging smoothly across BFSK frequency switches (panels 1–2, 100 μ s) and QPSK phase switches (panels 3–4, first 20 μ s).

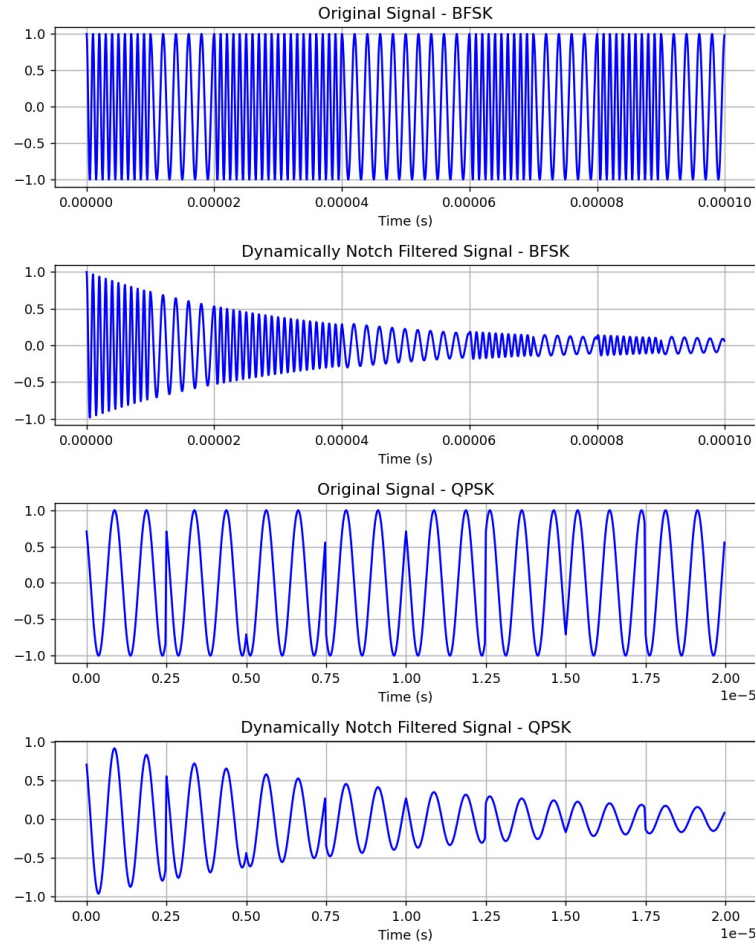


Figure 6. Reproduced paper Fig. 12: the dynamic notch filter decaying smoothly across the same BFSK and QPSK transitions.

These two figures show the charge-preservation claim directly. For BFSK, the dynamic bandpass envelope rises steadily toward unity over all ten symbols with no visible reset at the frequency changes (Fig. 5). For QPSK the envelope also keeps growing through the phase transitions. The small kinks at the symbol boundaries come from the output phase restoration $e^{j\varphi}$ being applied to the real part, and are not a loss of charge. The notch filter (Fig. 6) behaves in the opposite way: the output starts near 1 and decays smoothly, so the tracked signal is removed gradually instead of being excited again at every transition.

4.4 BFSK Dynamic Bandpass and Notch Filtering

Table 2. BFSK simulation parameters.

Parameter	Value (as used in the script)
Sampling rate, f_s	16 Msps
Carrier frequencies, f_1 / f_2	1.0 MHz / 0.5 MHz
Symbol period / count	10 μ s / 10 symbols (160 samples each)
Filter time constant, τ	31.2 μ s (feedback $ b = 0.997999$)
τ / symbol period	3.12
Noise	None (ideal analytic signal)

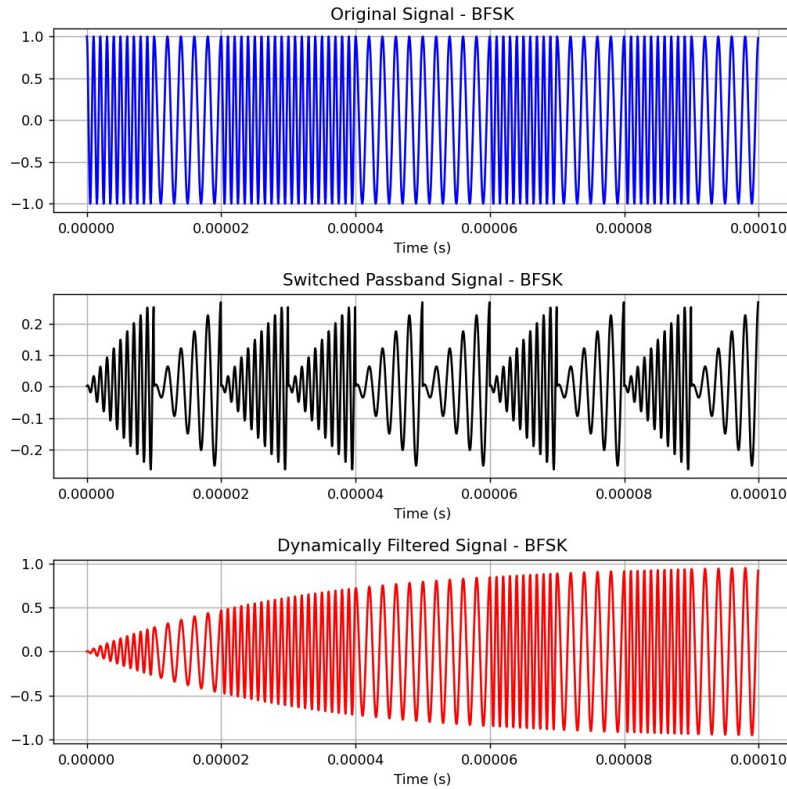


Figure 7. Reproduced paper Fig. 13: BFSK original signal (top), switched bandpass with state reset every symbol (middle) and dynamic bandpass (bottom).

The peak output of the switched filter is about the same in every symbol (0.264–0.269). Each symbol restarts the charging from zero, and $10\ \mu\text{s}$ is only about a third of the $31.2\ \mu\text{s}$ time constant (theory: $1 - e^{-10/31.2} = 0.274$). The peak of the dynamic filter keeps growing, $0.264 \rightarrow 0.453 \rightarrow 0.608 \rightarrow \dots \rightarrow 0.9505$ over the ten symbols, and follows the analytic curve $1 - e^{-t/\tau}$ ($0.274 \rightarrow 0.473 \rightarrow \dots \rightarrow 0.959$) to within about 1%.

At the nine symbol boundaries, the complex envelope of the dynamic filter changes by only 0.12% on average (0.52% at the first boundary, falling to 0.01% by the ninth). The envelope of the switched filter collapses by 99.3% at every boundary (from 0.274 to 0.002). The small jump that remains in the dynamic filter happens because our BFSK signal is phase-discontinuous at the switch, so a tiny part of the stored charge is slightly out of phase with the new tone.

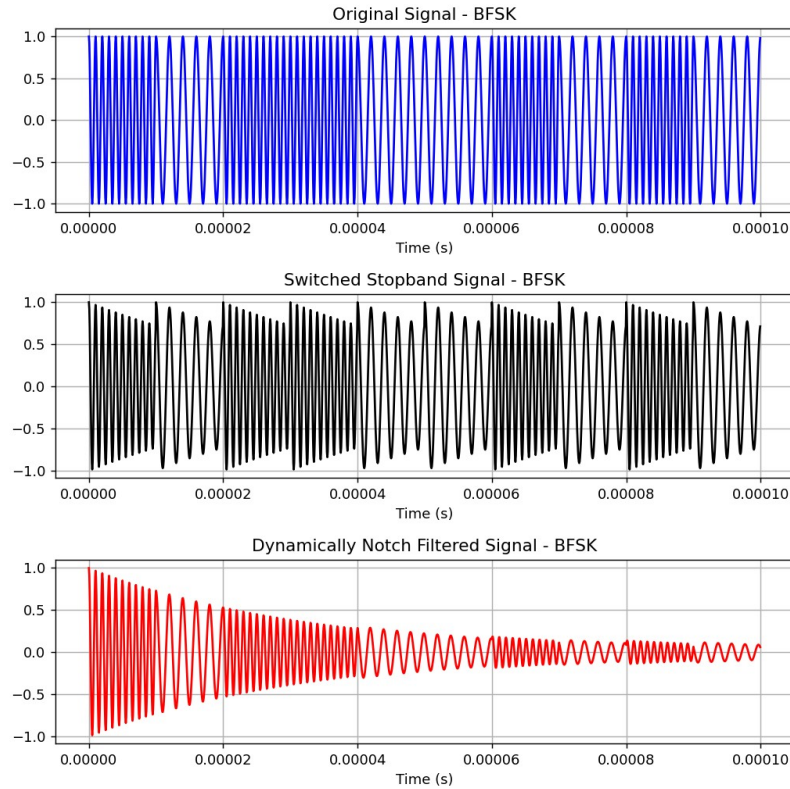


Figure 8. Reproduced paper Fig. 14: BFSK original (top), switched notch (middle) and dynamic notch (bottom).

For the notch filters, the switched version removes almost nothing. Its RMS output is 0.608 in every symbol (the input RMS is 0.707) and its peak is 0.999, because the restart at each symbol brings back the start-up transient of the high-pass prototype. The dynamic notch RMS instead decays $0.608 \rightarrow 0.446 \rightarrow 0.325 \rightarrow 0.236 \rightarrow 0.188 \rightarrow 0.136 \rightarrow 0.116 \rightarrow 0.091 \rightarrow 0.086 \rightarrow 0.075$ over the ten symbols. In the last symbol this is 10.6% of the input RMS, an attenuation of about 19.5 dB, and it would keep improving with a longer record. This is the “continuous decay” behaviour: the notch keeps removing the tracked tones across the frequency hops without restarting.

4.5 QPSK Dynamic Bandpass and Notch Filtering

Table 3. QPSK simulation parameters.

Parameter	Value (as used in the script)
Sampling rate, f_s	32 Msps
Carrier frequency, f_c	1.0 MHz
Symbol period / count	2.5 μ s / 28 symbols (80 samples each)
Filter time constant, τ	10.4 μ s (feedback $ b = 0.996999$)
τ / symbol period	4.16
Noise	None (ideal analytic signal)

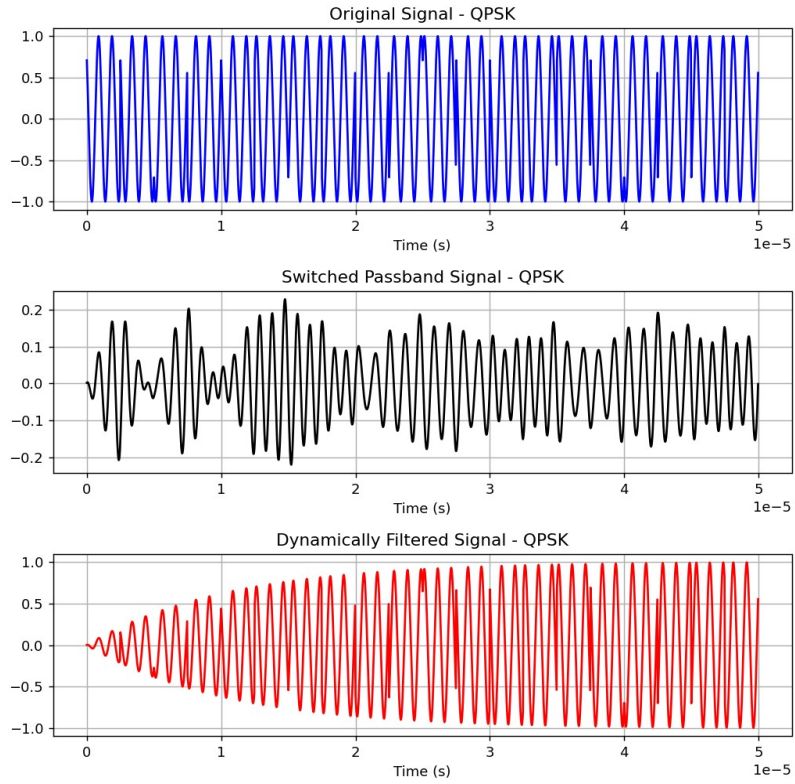


Figure 9. Reproduced paper Fig. 16: QPSK original (top), fixed narrowband filter with no phase correction (middle, labelled “switched” in the script) and dynamic phase-nullification filter (bottom), first 50 μ s.

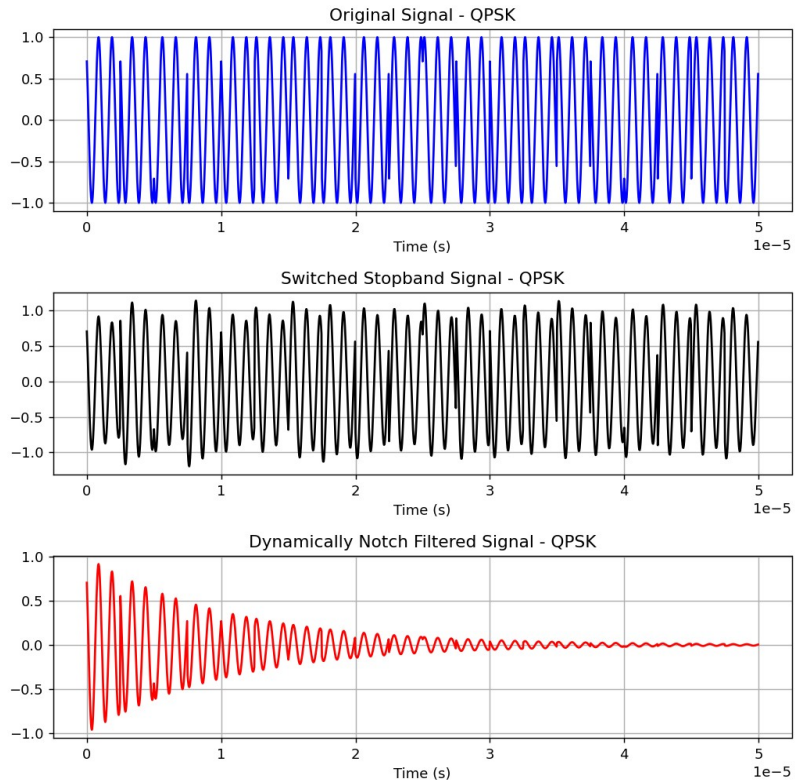


Figure 10. Reproduced paper Fig. 17: QPSK original (top), fixed notch filter (middle) and dynamic notch filter (bottom), first 50 μ s.

Table 4. QPSK comparison of the fixed and dynamic filters (complex envelope over the full 70 μ s).

Metric	Fixed filter (no phase correction)	Dynamic filter
Mean envelope, second half of record	0.151	0.990
Coefficient of variation, second half	0.404	0.009
Envelope at end of record	0.117	0.9988 (theory 0.9988)
Notch output RMS, first symbol – last symbol	0.625 – 0.706 (no suppression)	0.625 – 0.0009 (about –57.9 dB relative to input)

Without phase correction, the narrowband filter cannot hold a charge. At every phase jump the charge left from the previous symbol fights the new symbol, so the envelope stays small (mean about 0.15) and varies a lot (CV about 0.40). With phase nullification, the dynamic filter sees an unmodulated sinusoid and charges exactly as $1 - e^{-t/\tau}$. It reaches 0.9988 at 70 μ s, equal to the analytic value, with an envelope ripple below 1% (CV 0.009). The dynamic notch drives the tracked QPSK signal down to 0.0009 RMS by the last symbol (about –58 dB relative to the input), whereas the fixed notch keeps passing 0.57–0.76 RMS because it cannot null a signal whose phase keeps changing. Note also that, since our signals are ideal, this version reaches the clean, near-unity sinusoid shown in the paper’s Fig. 16(c).

4.6 Time-Constant Validation Across All Cases

Table 5. Time constants and bandwidths implied by the feedback coefficients, $|b| = e^{-1/(\tau f_s)}$.

Case	Time constant set in script	Resulting $ b $	$\Delta f = 1/(2\pi\tau)$	Paper’s $\Delta f, \tau$
BFSK bandpass / notch	31.2 μ s	0.997999	5.10 kHz	10 kHz, 31.2 μ s
QPSK bandpass / notch	10.4 μ s	0.996999	15.30 kHz	30 kHz, 10.4 μ s
Frequency-response demo	31.83 μ s (from cutoff 0.005)	0.969072	5.0 kHz	—

In this script $|b|$ is set directly from the paper’s time constants through Eq. (17), so the BFSK and QPSK filters have $\tau = 31.2 \mu$ s and 10.4μ s exactly. The bandwidths implied by $\Delta f\tau = 1/(2\pi)$ are 5.10 kHz and 15.30 kHz, which is exactly half of the 10 kHz and 30 kHz quoted in the paper. The two-sided values $2 \times 1/(2\pi\tau) = 10.2$ kHz and 30.6 kHz match the paper to about 2%. So the paper’s Δf is best read as the full edge-to-edge bandwidth, while Eq. (1) gives the half-width. This agrees with Section 4.1, where the 4.9 kHz prototype cutoff gives a 9.8 kHz edge-to-edge width.

Two more checks confirm the recursion is implemented correctly. The dynamic QPSK envelope at 70 μ s equals $1 - e^{-70/10.4} = 0.9988$ exactly, and the BFSK envelope at 100 μ s (0.955) is within 0.5% of $1 - e^{-100/31.2} = 0.959$.

5. Critical Evaluation

5.1 What Makes This Method Unique

The main idea of the method is to split a filter’s complex coefficient into a magnitude, which fixes the bandwidth and time constant, and a phase, which fixes the centre frequency or the instantaneous phase reference. Since only the pole magnitude decides the settling time, a receiver that knows the frequency or phase trajectory of the signal can retune the filter phase instantly, symbol by symbol, without paying the usual settling-time penalty. This turns the normal Q-versus-settling-time trade-off into two separately

controllable quantities. It is a useful conceptual contribution, even though the paper says clearly that the filter is an idealized system-level abstraction and not a circuit that can be built directly.

5.2 Strengths of This Reproduction

- **One general engine.** A single time-varying filter routine (per-sample weights, feedback, restoration and reset period) generates every bandpass, notch, switched, fixed and dynamic variant. The code stays short and each comparison is like-for-like.
- **Exact analytic validation.** Because the signals are noise-free, outputs can be compared with closed-form charging curves. The QPSK dynamic envelope matches $1 - e^{-t/\tau}$ to four decimal places.
- **Paper’s own time constants.** $|b|$ is set from $\tau = 31.2 \mu\text{s}$ and $10.4 \mu\text{s}$, so there is no ambiguity about which bandwidth convention is used.
- **Quantitative metrics.** Boundary envelope changes, per-symbol peaks, RMS suppression and coefficients of variation back up the visual comparisons.

5.3 Challenges and Limitations

- **No noise.** The paper’s BFSK/QPSK simulations use 10 dB SNR, but our script uses ideal signals. The noise-bandwidth benefit of the PSDF (paper Eqs. 18–20) is therefore not measured, and the spectra of paper Figs. 15 and 18 are not reproduced.
- **Missing application studies.** The AM-interferer test (Fig. 19), the full-duplex co-site leakage simulation (Fig. 21) and the mismatch-sensitivity curve (Fig. 22) are not in this script.
- **Different “switched” baselines.** For BFSK the switched filter restarts from zero every symbol, which is a worst-case baseline compared with a real filter bank where the inactive filter free-decays. For QPSK the comparison filter is a fixed filter with no phase correction and no reset. The plot labels follow the paper’s titles, but the two baselines are not the same kind of filter.
- **Short records, single realization.** The BFSK record has only 10 symbols and the QPSK record 28, each with a fixed symbol sequence. The BFSK notch has not fully settled (residual 10.6%), and the numbers could change with other sequences.
- **Phase-discontinuous BFSK.** The BFSK signal is built as $\exp(j2\pi f(t)\tau)$ with absolute time, so its phase jumps at each hop. This causes the small (about 0.1–0.5%) envelope jump of the dynamic bandpass at the boundaries instead of exactly zero.

5.4 Possible Sources of Deviation

Apart from the missing noise, the frequency responses come from a 2048-point FFT of the impulse response, so the peak and null positions are quantized to the 0.49 kHz bin spacing (100.1 kHz instead of 100 kHz) and the –3 dB widths are accurate only to about one bin. The paper’s figures were made in Simulink/MATLAB, so the exact plotted shapes (for example the BFSK input phase at the start) can differ slightly from ours. Both the paper and this reproduction leave out circuit-level non-idealities such as switching transients, timing jitter and pump phase noise. Every comparison here is therefore at the level of the idealized mathematical model, and we make no claim about a physically realizable circuit.

5.5 How the Results Could Be Improved

- Add the 10 dB white (Hilbert-paired) noise of the paper, compute the output SNR and noise-bandwidth gain, and average over many random seeds to get confidence intervals.
- Add FFT spectra of the dynamic bandpass and notch outputs to reproduce paper Figs. 15 and 18.
- Add the AM-interferer test and the full-duplex leakage-cancellation block, and get the mismatch-sensitivity curve (Fig. 22) from a Monte-Carlo sweep with artificially delayed coefficient switching.

- Replace the BFSK hard-reset baseline with a two-filter bank whose inactive branch free-decays, and use longer records so the notch reaches steady state.
- Sweep the time constant (for example τ/T_s from 0.5 to 10) to see how the charging speed and noise bandwidth depend on the filter bandwidth.

6. Software and Implementation Details

All simulations were written in Python 3. NumPy was used for the array arithmetic and the sample-by-sample recursion, SciPy (`scipy.signal.lfilter`) for the impulse responses in the frequency-response demos, and Matplotlib for every figure. No special DSP or communications toolbox was used: each complex IIR recursion in Section 4 was coded directly from the paper's difference equations inside `time_varying_filter`. The script uses no random numbers, so every figure and metric in this report can be reproduced exactly by running it. The quantitative metrics in Section 4 (per-symbol peaks and RMS, boundary changes, coefficients of variation, bandwidths) were computed from the same arrays that the script plots. The full script (`DSP_OEL_CODE.py`) is listed in Section 3.5 and supplied as a separate file.

7. Conclusion

In this project we reproduced the core theory and time-domain results of a 2026 IEEE Access paper on Phase-Sensitive Dynamic Filters, using a short Python script written from scratch. We coded the paper's single-pole complex IIR recursions for dynamic bandpass filtering, dynamic notch filtering and the phase-nullification method for phase-shift-keyed signals, and compared them against switched and fixed filters. The results confirm the paper's main claim for both BFSK and QPSK: changing only the phase of a coefficient, and not its magnitude, lets the filter retune in step with the symbols without losing its stored state.

The dynamic bandpass filter changed its envelope by about 0.1% at BFSK symbol boundaries, against about 99% for a restarted filter. For QPSK it reached the analytic charging limit of 0.9988, while an uncorrected fixed filter stayed at about 0.14. The dynamic notch removed the tracked signal by about 19.5 dB (BFSK) and 58 dB (QPSK), where the baselines removed almost nothing. Setting the pole magnitude directly from the paper's time constants reproduced them exactly and showed that the paper's bandwidth figures are edge-to-edge values.

This version of the work is narrower than the paper. It is noise-free, it has no noise spectra, AM-interferer test, full-duplex study or mismatch-sensitivity analysis, and its "switched" baselines differ between BFSK and QPSK. We list these limits in Section 5 together with concrete ways to improve them. Beyond the paper itself, the project reinforced several DSP ideas from the course: turning a transfer function derived with the z-transform into a working recursive filter, seeing how pole magnitude and pole phase separately control settling time and centre frequency, and checking simulated results against both closed-form theory and a published reference.

References

- [1] S. C. Chen, K. D. Bhakta, I. P. Roberts, and Y. E. Wang, "Phase-Sensitive Dynamic Filters: A Conceptual Model for Energy-Preserving, Instantaneously Tunable, Symbol-Level Signal Processing Using Complex IIR Filters," *IEEE Access*, vol. 14, pp. 61414–61429, 2026, doi: 10.1109/ACCESS.2026.3685577.
- [2] S. C. Chen, "Phase-sensitive dynamic filtering with time-varying transmission line filters," M.S. thesis, Dept. Elect. Comput. Eng., Univ. of California, Los Angeles, CA, USA, 2023.
- [3] F. G. Stremler, *Introduction to Communication Systems*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1990.

- [4] S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*. San Diego, CA, USA: California Technical Publishing, 1997.